

2025-2 / IIC2173 - E1 | PPE Properties Market Async

aka. Procesamiento de Pagos como Eventos

Fecha de entrega: 05/10/2025 - 3.5 Semanas

Después de un marcado éxito en la obtención y recopilación de visitas a propiedades, su consultora favorita Legit Business los contrata full-time (con beneficio de taca-taca, pizza y mesa de ping-pong en vez de seguro de salud), y con una promesa de barra de snacks libre les pide que ahora, en grupos de trabajo escogidos por un algoritmo *state of the art*, pasen a una versión mucho más poderosa de su plataforma.

El mercado inmobiliario vive de las agendas que suelen seguir un FIFO, que suelen ser reservadas de una forma muy rápida y poco amigable para un consumidor promedio. Para mejorar esto generarán una plataforma que cobrará por agendar la visita a la propiedad, permitiendo que los tengan que aceptar en su visita y permitir a los consumidores revisar todas las propiedades que les interesen.

Objetivo

Asumiendo una cantidad finita de visitas a propiedades a distintos precios, deben extender su servicio principal para que sea capaz de conectarse a un canal del broker y publicar sus solicitudes de compra de agendas de visita. Para esta entrega asumiremos que siempre envían el monto correcto y que pedirán el stock correcto.

Propiedades

Se conserva el canal de la E0 que emite las propiedades: `properties/info`. Recuerden que las propiedades que se indican acá son la posible visita que pueden agendar desde su plataforma. Este mensaje tiene el siguiente *schema*

```
{
  "name": string,
  "price": float,
  "currency": string,
  "bedrooms": string,
  "bathrooms": string,
  "m2": string,
  "location": string,
  "img": string,
  "url": string,
  "is_project": boolean,
  "timestamp": string (formato de fecha ISO 8601)
}
```

Solicitudes de Compra de Agendamientos

Para hacer la compra, **deben hacerlo al 10% precio de arriendo informado**. Para esto se les pedirá que se conecten a un nuevo canal del broker MQTT para que puedan publicar eventos de solicitud de compra, y que luego reciban por otro canal las validaciones de las solicitudes de compra correspondientes. Esto permite un proceso de compra completamente asíncrono, que pueda manejar muchas solicitudes.

Solicitud de Compra

El usuario que desee agendar una visita usando su dinero, deberá usar su servicio para comprarla. En su interfaz, debe existir una forma de recargar un wallet con dinero (pueden agregar sumas arbitrarias) para agendar de forma amigable y sencilla, con la información del estado actual de la propiedad (visitas disponibles, precio, etc).

Una vez que su usuario les indique que quiere agendar la visita, intentará reservarla generando una intención de pago que **deberán publicar en el broker en el canal `properties/requests`**, con el siguiente formato:

```
{
  "request_id": uuid,
  "group_id": string, //número de grupo
  "timestamp": string, //ISO 8601
  "url": string,
  "origin": 0, //Por ahora es 0 siempre
  "operation": "BUY" // Por ahora es "BUY" siempre
}
```

En el body que enviará, `request_id` será un [UUIDv4](#) de su solicitud que manejarán ustedes que para cada una deberá ser distinta (en la sección de enlaces útiles hay links que les pueden servir), y `group_id` su número de grupo.

Como todos los grupos mandarán mensajes por este canal, deberán estar escuchándolo para recibir las solicitudes de los otros grupos. Así, si un grupo tiene una compra exitosa, deberá descontar las posibles visitas de esa propiedad en particular.

Respuesta de solicitud

Luego de esto, deberán escuchar el canal `properties/validation` para saber si su solicitud se pudo completar con éxito.

```
{
  "request_id": string,
  "timestamp": string, //ISO 8601
  "status": enum("ACCEPTED", "OK", "error", "REJECTED"),
  "reason": string
}
```

Si reciben una respuesta de que esa transacción es válida, entonces deben marcarla como completada. En otro caso (solicitud inválida), deberán marcarla como que no se logró. Para esta entrega una vez que se reciba esta validación se terminará el flujo.

Los status pueden ser

- "ACCEPTED": Se aceptó el pago y la solicitud es aceptada
- "OK": Se recibió la petición pero no se ha procesado
- "error": Su petición tiene algún error de formato JSON o de los campos involucrados, como un symbol no existente o un error de tipo
- "REJECTED": Su petición no pudo ser completada, por falta de horarios de visita para la propiedad.

Event log

Deben guardar todos los eventos que introduzcan un cambio en el universo de visitas disponibles:

- Si llegó un nuevo evento de visita
- Ustedes agendaron una visita
- Otro grupo agendó una visita exitosamente

Backlog (a.k.a wishlist)

Si un requisito está marcado como *Esencial*, el no cumplirlo en un grado mínimo (al menos un punto salvo que se especifique) reducirá la nota máxima a *4.0*.

Requisitos funcionales (15 pts)

- **RF01 (2 pts) (Esencial)** Sus usuarios deben poder registrarse en la plataforma con datos de contacto y un correo electrónico.
- **RF02 (2 pts) (Esencial)** Los usuarios deben poder ver una lista de propiedades disponibles para visitas dado un filtro de búsqueda sobre todos los campos descriptivos disponibles pedidos en los filtros de la E0. Esta lista debe estar paginada.
- **RF03 (2 pts)** Debe haber una vista donde se pueda ver el detalle de cada propiedad, incluyendo la cantidad de visitas disponibles y dar la opción de compra de agendamiento, la que debe costar un 10% del precio de arriendo.
- **RF04 (2 pts)** Deben poder mostrarle a su usuario las visitas agendadas y si su solicitud se completó correctamente.
- **RF05 (3 pts) (Esencial)** Al realizar una compra se deberá enviar la solicitud por el canal `properties/requests` y esperar la respuesta de si es válida por el mismo canal.
- **RF06 (4 pts) (Esencial)** Deberán estar escuchando los canales de `properties/info`, `properties/requests` y `properties/validation` y tomar las acciones correspondientes por cada evento. Esto incluye grabar cada uno de los eventos especificados previamente en su base de datos. Este punto se evalúa completo, no puede tener medio puntaje.

Ejemplo de flujo a nivel de usuario

1. Un usuario ingresa en la plataforma con credenciales creadas anteriormente a su aplicación.
2. Luego va a una lista de propiedades con capacidades de filtrado.
3. Para comenzar, escogerá una propiedad cualquiera y revisará el detalle de los precios de agendamiento de visitas.
4. Se descontará un agendamiento de la cantidad de visitas totales disponibles para esa propiedad en sus registros para reservarlos mientras se valida. Se verá si puede pagarlo y se le creará una solicitud de compra avisando que su compra está en proceso.
5. (Validación) Luego de un periodo de tiempo corto, el usuario revisará su listado de visitas y su compra se marcará de *en proceso* a *completada correctamente*, o no, de acuerdo a lo entregado en el canal de validaciones. Si resulta inválido la visita se devuelve a disponibilizar al registro que lleva su sistema.

En caso de que ustedes reciban una solicitud de otro grupo por el broker, deben descontar esas visitas de igual manera ya que están reservados por ese grupo mientras se valida el pago. Una vez recibida la validación si es correcta lo mantienen igual, y si no, devuelven la posible visita a sus registros. Deben manejar posibles casos de inconsistencia en sus entregas.

Requisitos no funcionales (39 pts)

Requisitos no funcionales

- **RNF01 (6 pts) (Esencial)** Deben usar un formato de Backend-Frontend separado: una API con respuestas JSON y un frontend. Esto es muy importante puesto que es crítico para las siguientes entregas. El Frontend debe ser una SPA con un Framework que permita exportar el build de su frontend como artefactos estáticos basado en React o Vue.
 - Deben usar dos repositorios distintos para manejar su backend y su frontend
- **RNF02 (2 pts) (Esencial)** Sus aplicaciones en backend deben estar cada una en un contenedor Docker distinto. Debe coordinarse el levantamiento mediante docker compose.
- **RNF03 (2 pts) (Esencial)** Deben tener configuradas *Budget alerts* en la cuenta que ocupen como grupo, para no alejarse del Free tier de AWS y que se den cuenta si les cobran para que cierren esos servicios.
- *RNF04 (6 pts) (Esencial)*: Su API debe estar detrás de una AWS API gateway tipo REST o HTTP, con los endpoints declarados en esta. Debe asociarse un subdominio a esta (e.g. <https://api.miapp.com>) y debe tener CORS correctamente configurado.
- *RNF05 (3 pts) (Esencial)*: Su app debe ofrecer su backend y frontend utilizando HTTPS.

- **RNF06 (5ptos) (Esencial):** Deben implementar un **servicio de autenticación/autorización** (auth). Este servicio puede ser en base a un servicio de terceros como Auth0 (recomendado), Cognito o pueden hacerlo ustedes. Este RNF requiere que ustedes extraigan toda la lógica de los usuarios de la app principal y la trasladen a el servicio hecho por ustedes o el externo. Recomendamos fuertemente usar el modelo OAuth o como mínimo intercambiar tokens JWT con la audiencia e issuer correctos.
 - Si hacen un servicio ustedes desde 0, tienen un **bonus de 5ptos**. El servicio incluye gestión de tokens JWT con OAuth2 con HMAC, gestión de la información de usuarios, una gestión segura de los tokens a nivel de frontend, y **debe correr como un contenedor/servicio aparte**.
- **RNF07 (3ptos):** Su frontend debe estar desplegado en S3 con una distribución Cloudfront.
- **RNF08 (3ptos):** Su API Gateway debe poder usar al servicio del RNF06 para autenticar a los usuarios directamente, es decir que antes de enviar la request a su API, API Gateway verifica que el token sea correcto. Dentro de API Gateway deben crearle un Custom Authorizer si usan tipo REST para poder autenticar sus requests previos a mandarlos a su API.
- ***RNF09 (5ptos)*** Deben implementar monitoreo utilizando un proveedor de monitoreo SaaS. Se recomienda New Relic puesto que su free tier es generoso. Deben mostrar que pueden recibir métricas en su dashboard principal de new relic. No es necesario que hagan algo con estas.
 - Debe habilitarse el monitoreo de infraestructura
 - Debe habilitarse el monitoreo de aplicación
- ***RNF10 (3ptos)*** Deben implementar las siguientes medidas de resiliencia
 - Sus containers deben levantarse al caer
 - Deben hacer retry de cada petición al broker usando un delay fibonacci.
 - Deben dar la posibilidad de hacer retry de requests fallidas al usuario, una sola vez.
- ***RNF10 (1pto)*** Al querer generar un pago de un arriendo en UF deberán obtener el valor en peso Chileno del 10% de este según su valor del día para generar el intento de pago. Para esto pueden buscar el valor diario de la UF usando la API que les mostramos en enlaces útiles o mediante otra forma que les entregue el valor real.

Documentación (6ptos)

Todos estos documentos los deben dejar dentro de su repositorio de Github en una carpeta `/docs`.

- ***RDOC01 (3ptos)*** Deben crear un diagrama UML de componentes de la entrega, con **explicaciones y detalle** sobre el sistema. Esto deben tenerlo para la fecha final de entrega.
- ***RDOC02 (2ptos)*** Deben documentar los pasos necesarios para replicar la instalación y seguimiento de su flujo de Monitoreo (que pasos siguieron y como se ve la información)
- ***RDOC03 (1ptos)*** Deben dejar una documentación de alguna forma de correr su aplicación en un ambiente local para propósitos de testeo (que instalar, que poner en el .env, como correr la app, etc).

Meta: Sobre la entrega

- **NO se revisaran entregas que no estén en la nube bajo ningún concepto.**
- **Deben tener todos los requisitos no variables de la E0 para esta entrega o NO se les corregirá.**
- **Para esta entrega y el resto del proyecto les pediremos que creen un repositorio personal y que luego integren a su ayudante respectivo con accesos de administrador.**

Debido a que esta entrega presenta una buena cantidad de *bonus*, la nota no puede sumar más de 8. Decidan con sabiduría que bonus les gustaría aprovechar o usar para compensar.

Al final de la entrega, la idea es que se pongan de acuerdo con su ayudante para agendar una hora y hacer una demo en vivo para su corrección. Se revisarán algunas features esenciales de código (conexión al broker). Pueden utilizar Github Copilot (recomendado) para multiplicar su productividad y ChatGPT (poco recomendado) para dudas de implementación, pero la información más completa la encontrarán en las documentaciones oficiales y preguntando a sus ayudantes / profesores. Si utilizan código copiado de una plataforma deben citarlo en el README.md, y pregunten a su ayudante antes de usarlo.

Descuentos

Está totalmente prohibido subir su archivo .env o .pem al repositorio, en caso de subir su archivo .env se les descontará 5 decimas, y en el caso del .pem no se corregirá su entrega.

Consideraciones

No se considerarán entregas:

- Con componentes que corran en sus computadores o servidores que no sean los básicos de AWS. Algunos ejemplos, los servicios de AWS serían:
 - EC2
 - VPC
 - IAM
 - S3
 - AWS API Gateway
 - CloudFront
 - Lambda
- Montadas en Heroku/Firebase/Elastic beanstalk/Lightsail/Netlify o similares.
- Que no estén documentadas.

Pueden utilizar cualquiera de los siguientes lenguajes. Los frameworks son los recomendados. Sugerimos que utilicen en cada grupo el lenguaje que todos sepan usar más o les parezca más sencillo de aprender. Los lenguajes en cursiva poseen ayudantes capaces de ayudarlos en su implementación de código.

- *Python*
 - FastAPI
 - Django
- *Javascript (js)*
 - Koa
 - Express

- NestJS (solo si tienen muuuucha experiencia, aprenderan mucho y ocuparán typescript)
- *Ruby*
 - Rails

En frontend, con el sistema de estado que prefieran

- *Javascript*
 - React
 - Vue
 - Next
 - Nuxt

Recomendaciones

- Comiencen la entrega lo antes posible, puesto que es más sencillo ir trabajando de a partes y seguro tendrán dudas. Se les dio plazo extra para que se adecuen a sus equipos de trabajo.
- Planifiquen con antelación: pregunten dudas o ambigüedades a sus ayudantes.
- **Ojo con los deploys a última hora, la maldición de la demo es muy real.**
- Ocupen el Free Tier de AWS, que tiene capacidad para todos estos requerimientos. Deberían usar los siguientes servicios:
 - **EC2:** AWS les proporciona una instancia t3.micro gratuita al mes.
 - **S3:** Tienen 5 GB de almacenamiento y 20000 solicitudes GET.
 - **RDS** (Opcional, Recomendado): Tienen 20GB y una instancia básica al mes.
 - **API Gateway:** 1 MM de llamadas al mes
 - **Lambda (Opcional):** Tienen 400K GB/s y 1 MM de solicitudes.
 - **EBS:** 30 GB al mes para almacenamiento de discos de sistema.
 - **Cloudfront:** 50 GB al mes de transferencia.
 - **Amazon SES:** 62000 mensajes salientes / mes.
- **NO** está planificado hacer devolución por uso de dolares en AWS. Para la entrega el free tier de AWS es suficiente para conseguir todos los puntos. En caso de utilizar dólares en su solución, el curso no puede hacerles devolución de estos bajo ninguna causa. Si tienen problemas con costos que no saben como detener hableslos con su ayudante designado.
- **Usen una cuenta nueva o de alguien que no tenga otras cargas en AWS, para evitar cargos por ahora** , además de usar una tarjeta prepago y los budget alerts de AWS para evitar costos oculto.
- Pueden usar librerías para generar el gráfico de precios semanales.

Sobre el puntaje

Grupal

La nota se calcula como:

$$E_1 \text{ grupal} = 1 + E_1 \text{ RF} + E_1 \text{ RNF} + E_1 \text{ RDOC}$$

Siendo $E_1 \text{ RF}$ el puntaje ponderado de los requisitos funcionales, y $E_1 \text{ RNF}$ el correspondiente a los requisitos no funcionales y $E_1 \text{ RDOC}$ de la documentación.

Individual

Segun el programa del curso⁵ , esto se evalua como:

$E_1 = 1 + ((E_1 \text{ grupal} - 1) * F_g)$ Donde F_g es un factor de coevaluación asignado por el grupo que va de 0 a 1.2. Para esto se enviará un form de coevaluación donde cada integrante deberá evaluar a sus compañeros de grupo con una puntuación entre 1 y 5.

No podrán asignarle 5 puntos a más de un compañero, y si lo hacen, se considerará que se entregó un máximo de 4 puntos a cada compañero .

De no realizar la coevaluación, asumiremos que se le entregó el mismo puntaje de coevaluación a cada integrante, es decir 4 puntos.

Atraso

Para esta entrega se les descontará 0.5 puntos en la nota máxima por horas Fibonacci con $F1 = 6$ y $F2 = 6$.

Se considerará como atraso cualquier modificación en features o implementación que tenga que ver solo con lo que se pide en esta entrega.

Fibonacci	Hora	Nota maxima
6	0:01 - 5:59	6.5
6	6:00 - 11:59	6
12	12:00 - 23:59	5
18	24:00 - 41:59	4.5
30	42:00 - 71:59	4
...	72:00 en adelante	1

Links útiles

- [API Gateway - Rest API](#)
- [Auth0 - Get Started](#)
- [Documentación de Auth0 con API Gateway](#)
- [Circle CI Blogs - CI para Django](#)
- Librerías UUID
 - [Python](#)
 - [NodeJS](#)
 - [Ruby](#)
- Linters
 - [Flake8 - Python](#)
 - [ESLint - JavaScript](#)
 - [RuboCop - Ruby](#)
- [Monitoreo de infraestructura con New Relic](#)
- [API Valor UF - mindicador.cl](#)

Apoyo

Cada grupo tendrá un ayudante asignado el cuál podrán elegir mediante un link que se les mandará oportunamente. Este ayudante está encargado de hacerles seguimiento y orientar sus dudas para responderlas ellos mismos y el equipo de ayudantes. Les recomendamos **fuertemente** que pregunten sus dudas a su ayudante de seguimiento puesto que conocen del proyecto o pueden dirigir sus dudas a otros ayudantes. Puede ser de enunciado, código o algún tópico que tenga que ver con el proyecto

Dado que cada ayudante puede tener pequeñas diferencias en sus correcciones, queda a criterio de este hacer relajos o hacer mas estrictas ciertas particularidades. Intenten tener un flujo de comunicación directo con sus ayudantes para aclarar posibles diferencias o decisiones de diseño, es válido negociar siempre y cuando no sea en base a requisitos esenciales.

Pueden usar el Slack del curso para dudas más rápidas. Usen el [canal #e1](#) para sus dudas.